

Ensemble Methods (Bagging and Boosting)

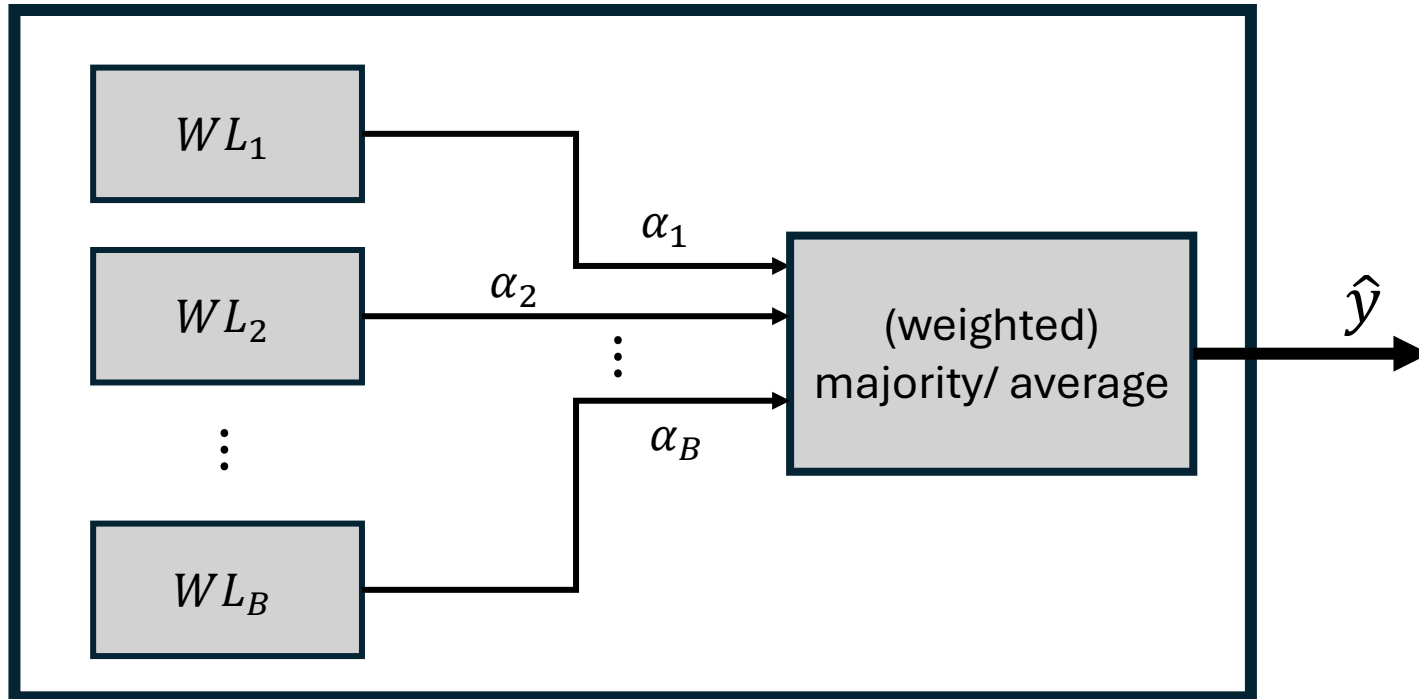
Avi Mohan

Today's Class

- What are **ensemble** methods?
- **Bagging**
 - Bradley Efron's Bootstrap Estimator
 - Bootstrap **A**ggregation (Bagging)
 - Random Forests (Leo Breiman)
- **Boosting**
 - AdaBoost

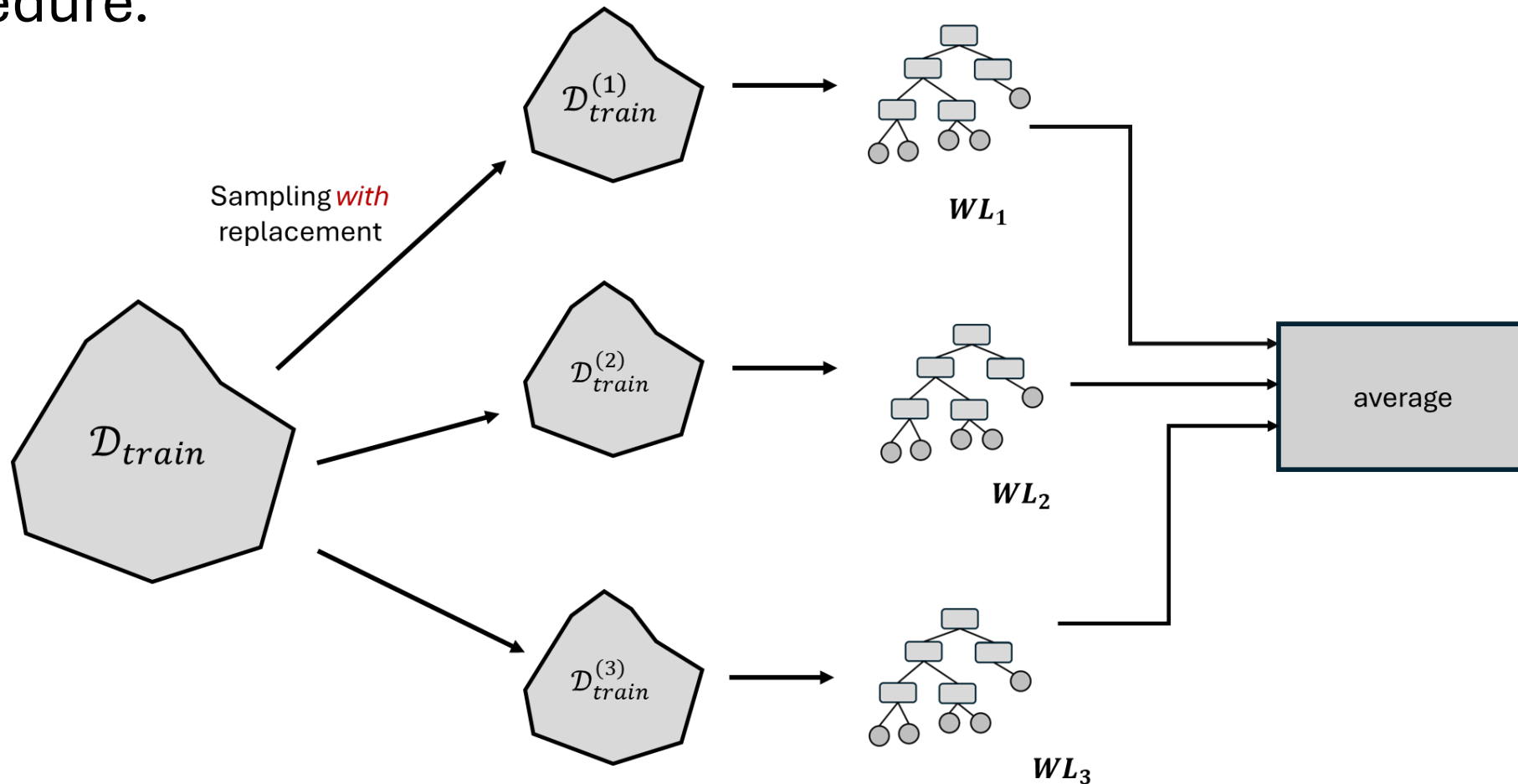
Ensemble Learning

- How many jellybeans/gumballs?
 - “Wisdom of the Crowds”
 - Source: <https://nrich.maths.org/problems/wisdom-crowd>



Bootstrap Aggregation (Bagging)

- General-purpose procedure to reduce *variance* of a statistical procedure.



Bootstrap Aggregation (Bagging)

- General-purpose procedure to reduce *variance* of a statistical procedure.

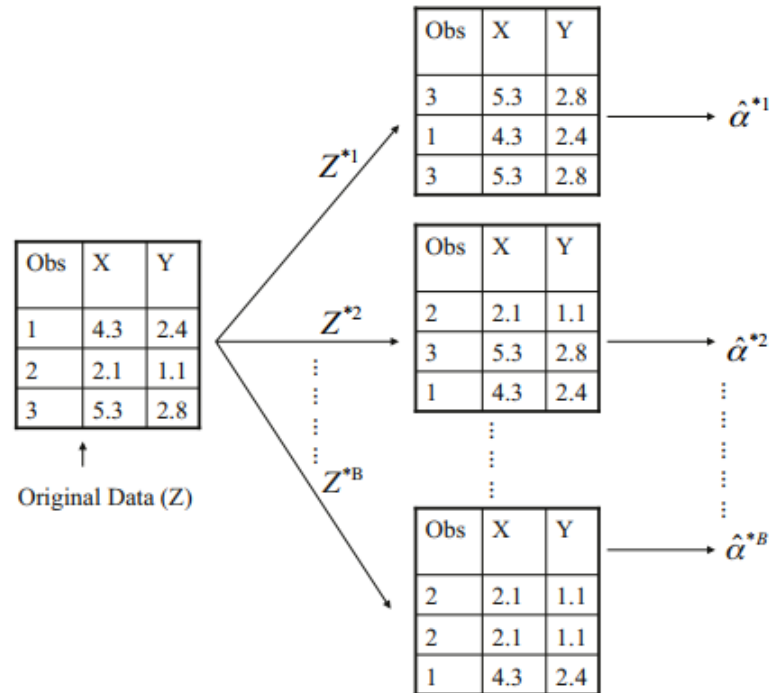


FIGURE 5.11. A graphical illustration of the bootstrap approach on a small sample containing $n = 3$ observations. Each bootstrap data set contains n observations, sampled with replacement from the original data set. Each bootstrap data set is used to obtain an estimate of α .

Bootstrap Aggregation (Bagging)

- General-purpose procedure to reduce *variance* of a statistical procedure.
- Given a set of B observations (random variables) $X_1, X_2, \dots, X_B$
 - Assume: observations are *independent* and identically distributed,
 - $\mathbb{E}X_1 = \mathbb{E}X_2 = \dots = \mathbb{E}X_B = \mu$
 - Variance = σ^2
- $\bar{X} = \frac{X_1 + X_2 + \dots + X_B}{B}$
 - Same mean $\mathbb{E}\bar{X} = \mu$
 - **Smaller variance = $\frac{\sigma^2}{B}$!**
- *Moral: averaging reduces variance.*

Bootstrap Aggregation (Bagging)

- Given a set of B observations (random variables) $X_1, X_2, \dots, X_B$
 - The observations are identically distributed
 - $\mathbb{E}X_1 = \mathbb{E}X_2 = \dots = \mathbb{E}X_B = \mu$
 - Variance = σ^2
 - Correlation coefficient = ρ

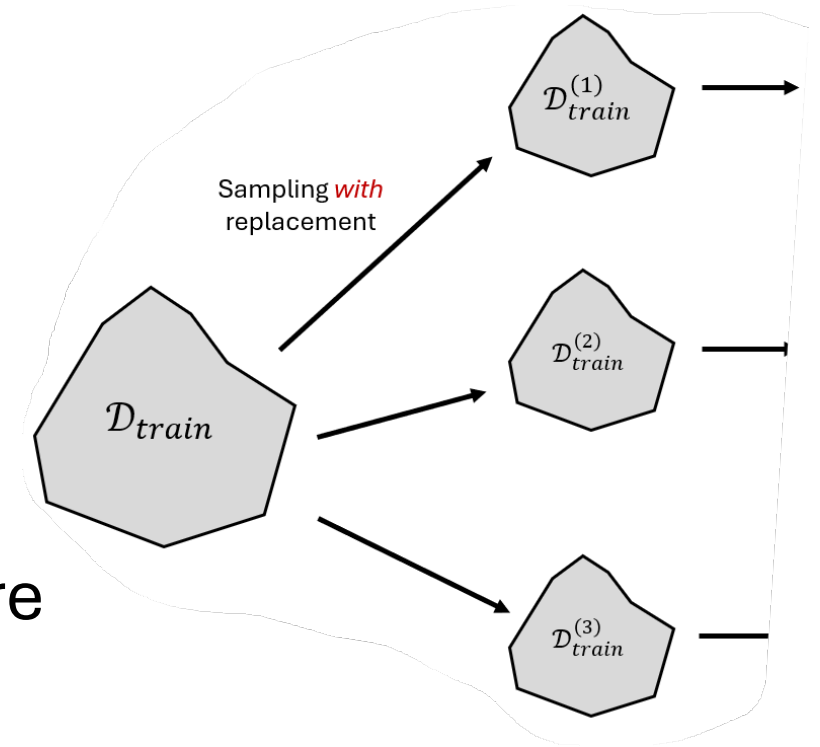
- $\bar{X} = \frac{X_1 + X_2 + \dots + X_B}{B}$

- Same mean $\mathbb{E}\bar{X} = \mu$

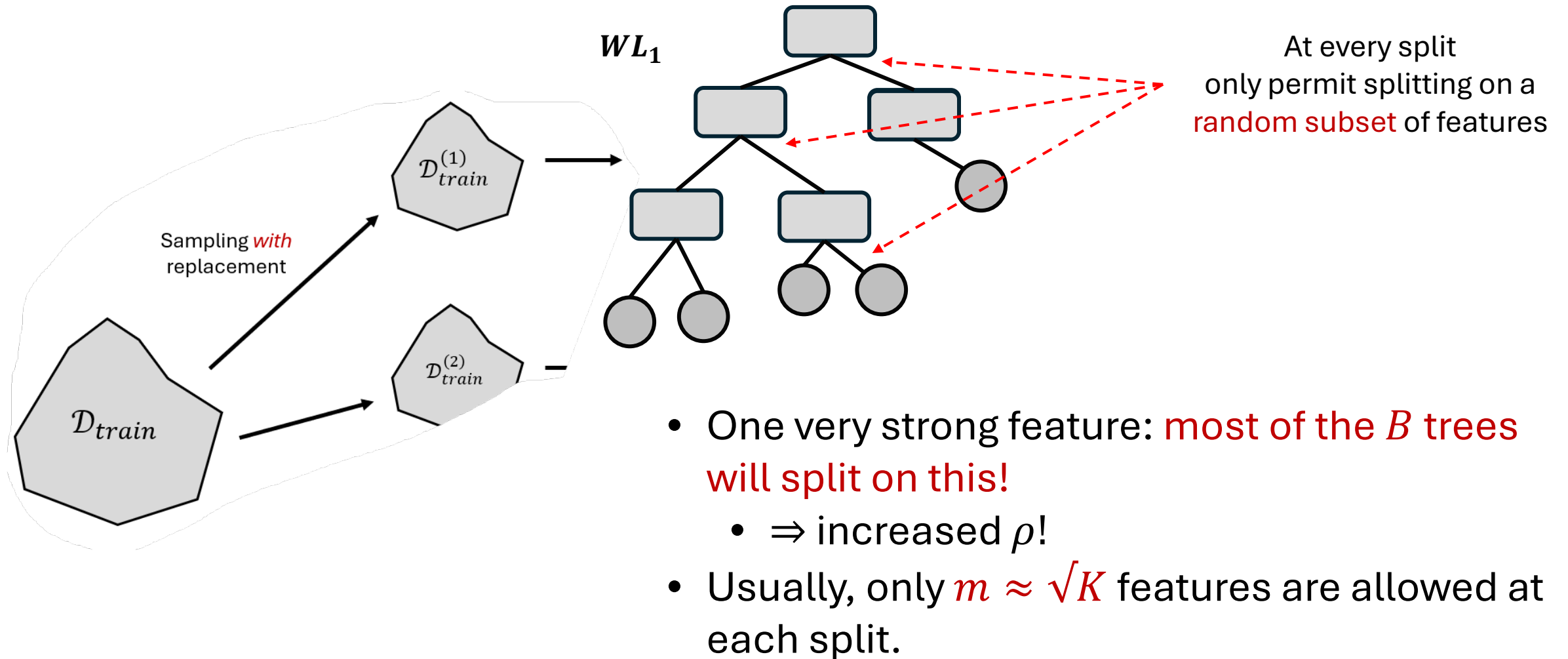
$$\text{var}(\bar{X}) = (1 - \rho) \frac{\sigma^2}{B} + \rho \sigma^2$$

Bagging

- $\mathcal{D}_{train}^{(i)} \subset \mathcal{D}_{train}$
 - Increased bias in each WL_i
- However, the large reduction in variance more than makes up for this increase!
- The trees are also grown deep and not pruned
 - Further decrease in bias
- Usually with decision trees, $B = 100$ works quite well.

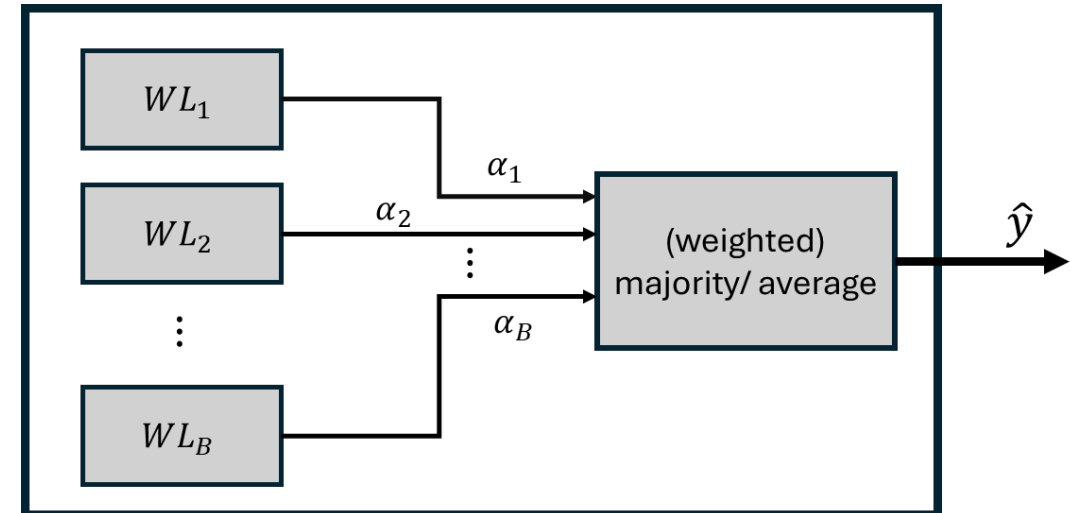


Random Forests



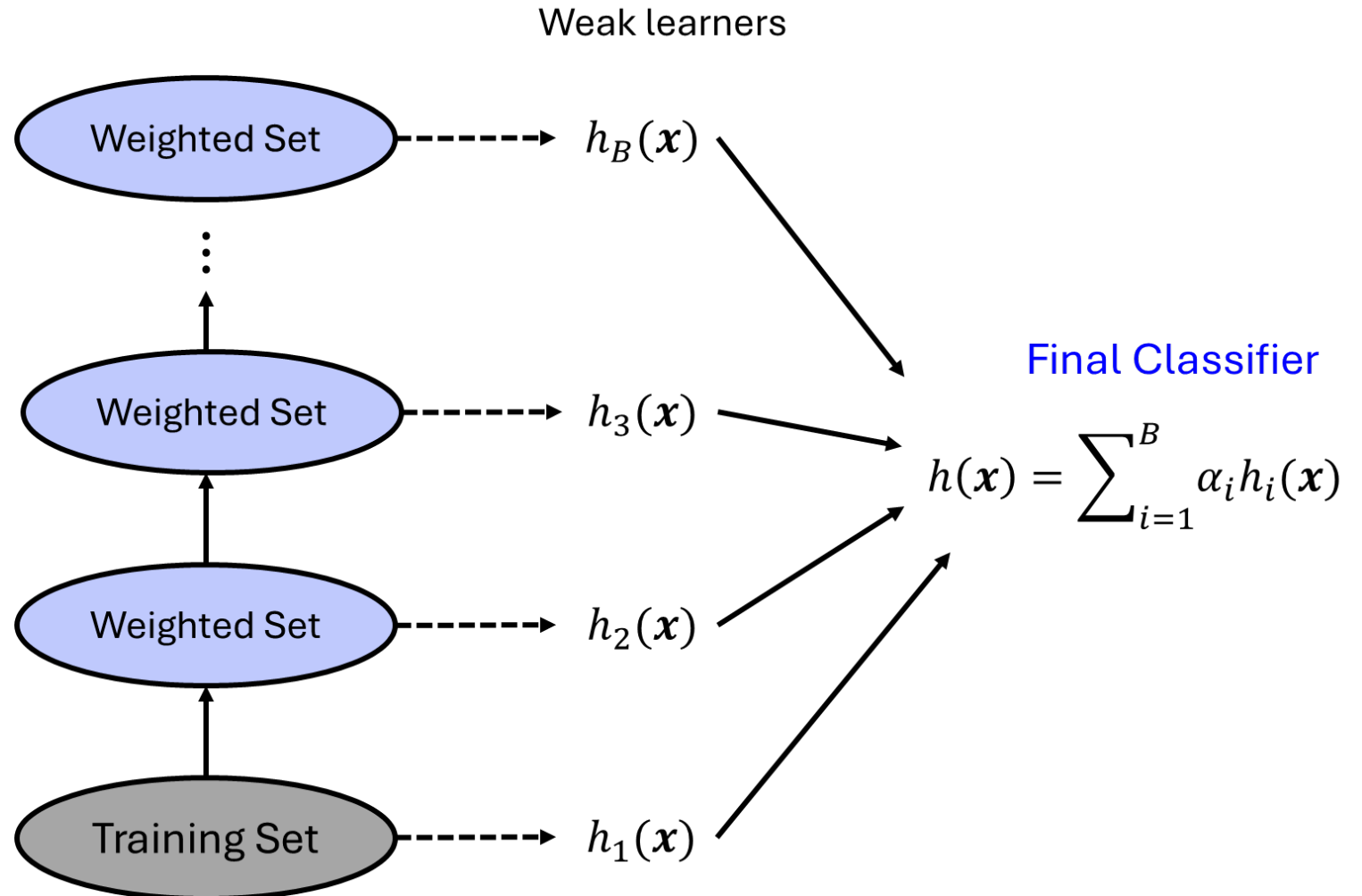
Today's Class

- What are **ensemble** methods?
- **Bagging**
 - Bradley Efron's Bootstrap Estimator
 - Bootstrap **A**ggregation (Bagging)
 - Random Forests (Leo Breiman)
- **Boosting**
 - AdaBoost

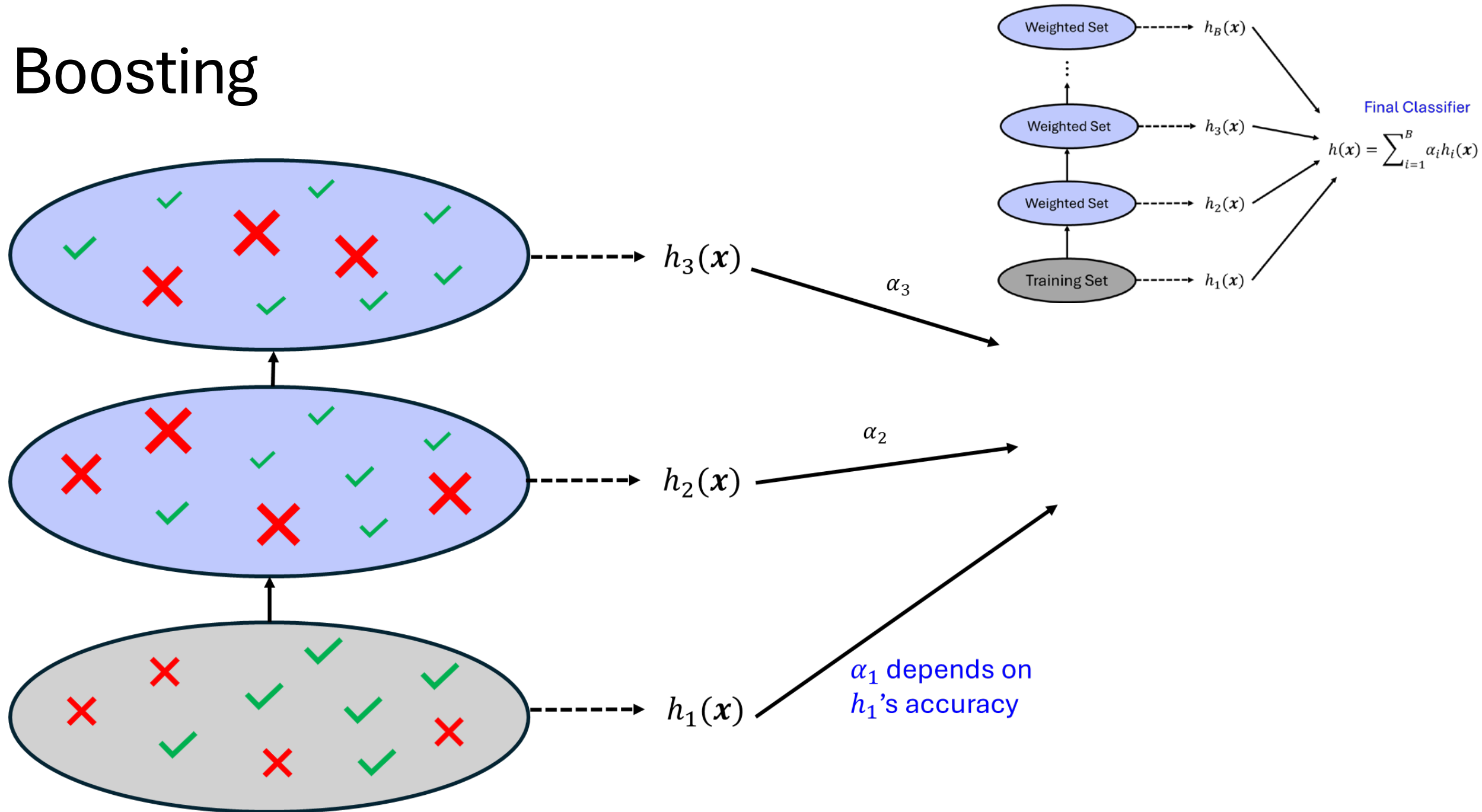


Boosting

- **Difference:** trees are grown *sequentially*.
 - Each tree is grown from previously grown trees.
- Sequentially apply weak classification algorithm to *modified versions of the data*.



Boosting



AdaBoost

- Inputs: $\mathcal{D}_{train} \equiv (\mathbf{x}_i, y_i)_{i=1}^{N_{train}}$, weights $w_i = \frac{1}{N_{train}}, i = 1, 2, \dots, N_{train}$, WL

- For $i = 1$ to B :

- $h_i(\cdot) \leftarrow WL(\mathcal{D}_{train}, (w_1, w_2, \dots, w_{N_{train}}))$
- For each training feature $\mathbf{x}$, the **predicted label** is the output of the current WL

$$\hat{y} = h_i(\mathbf{x})$$

- Compute weighted train error for the i^{th} WL :

$$err_i = \frac{\sum_{j=1}^{N_{train}} w_j \mathbb{I}\{y_j \neq \hat{y}_j\}}{w_1 + w_2 + \dots + w_{N_{train}}}$$

- Compute $\alpha_i = \log((1 - err_i)/err_i)$

- Compute new weights $w_j \leftarrow w_j e^{\alpha_i \mathbb{I}\{y_j \neq \hat{y}_j\}}, i = 1, 2, \dots, N_{train}$

AdaBoost

- Inputs: $\mathcal{D}_{train} \equiv (\mathbf{x}_i, y_i)_{i=1}^{N_{train}}$, weights $w_i = \frac{1}{N_{train}}, i = 1, 2, \dots, N_{train}$, WL
- For $i = 1$ to B :
 - $h_i(\cdot) \leftarrow WL(\mathcal{D}_{train}, (w_1, w_2, \dots, w_{N_{train}}))$
 - For each training feature $\mathbf{x}$, the **predicted label** is the output of the current WL
 $\hat{y} = h_i(\mathbf{x})$
 - Compute weighted train error for the i^{th} WL :
$$err_i = \frac{\sum_{j=1}^{N_{train}} w_j \mathbb{I}\{y_j \neq \hat{y}_j\}}{w_1 + w_2 + \dots + w_{N_{train}}}$$
 - Compute $\alpha_i = \log((1 - err_i)/err_i)$
 - Compute new weights $w_j \leftarrow w_j e^{\alpha_i \mathbb{I}\{y_j \neq \hat{y}_j\}}, i = 1, 2, \dots, N_{train}$
- Output: $h(\cdot) = \sum_{i=1}^B \alpha_i h_i(\cdot)$

Comments

- How weak is “weak?” The WL should beat a random classifier.
 - $(50 + \epsilon)\%$ accuracy is good enough!
- **Weak learners:**
 - Decision “stumps”
 - Naïve Bayes classifiers
 - Perceptrons
- Essentially: each iteration should
 - terminate fast
 - produce “good enough” learner

Comments

- How weak is “weak?” The WL should beat a random classifier.
 - $(50 + \epsilon)\%$ accuracy is good enough!
- **Weak learners:**
 - Decision “stumps”
 - Naïve Bayes classifiers
 - Perceptrons
- Essentially: each iteration should
 - terminate fast
 - produce “good enough” learner

Comments

- No need to strive for a (near) perfect classifier/regressor.
- Any weak learner can be boosted
- AdaBoost is very simple to implement
 - most of the complexity is in training the WL
- Generalization performance is quite good

Applications

- Collaborative filtering,
- Spam filtering,
- NLP/Speech recognition,
- Data Mining,
- and the list goes on...

The Netflix Challenge

- **Further reading:**

- Bell, Robert M., and Yehuda Koren. "**Lessons from the Netflix Prize Challenge.**" ACM SigKDD Explorations Newsletter 9, no. 2 (2007): 75-79.
- Bennett, J. and Lanning, S., 2007. **The Netflix prize.**
- Eliot van Buskirk, "**\$1 Million Netflix Prize So Close, They Can Taste It.**" WIRED magazine, June 17th, 2009.
- "The Netflix Prize," Wikipedia ([Link](#))
- Also, watch the mini-documentary produced by AT&T, "[*From the Labs: Winning the Netflix Prize.*](#)"